

Códigos de Control de Errores

(Unidad 2)

Guerra Jorge, Pesce Agustín, Villar Federico

22 de julio de 2026

Resumen

Esta unidad construye la base matemática sobre la que se apoya el resto del curso. Se parte de la observación de que el codificador combina símbolos mediante sumas y productos, y de que el decodificador necesita además las operaciones inversas, lo que obliga a dotar al alfabeto de una estructura algebraica: la de cuerpo finito. Se introducen los conceptos de grupo y de cuerpo, se estudia la aritmética sobre el cuerpo binario y sus polinomios, y se desarrolla la construcción de los cuerpos de Galois $GF(2^m)$ junto con sus propiedades básicas (conjugados, polinomios mínimos, elementos primitivos) y las técnicas de cómputo asociadas. La unidad cierra con un repaso de espacios vectoriales y matrices sobre cuerpos finitos, que constituye el lenguaje en el que se formulan los códigos de bloque lineales de la unidad siguiente. El tratamiento es formal, pero todos los resultados se orientan a su uso posterior en la construcción y decodificación de códigos.

1. Por qué hace falta el álgebra

En la unidad anterior se introdujo la idea de agregar redundancia estructurada al flujo de símbolos, y se vio con ejemplos sencillos (repetición, paridad, arreglos rectangulares) que esa redundancia permite detectar y corregir errores. Sin embargo, esos ejemplos se construyeron de manera artesanal. Para diseñar códigos potentes de forma sistemática, y sobre todo para decodificarlos con un costo razonable, se necesita una estructura matemática sobre el alfabeto de símbolos.

El razonamiento es el siguiente. El codificador combina símbolos de información mediante sumas y productos para producir los símbolos redundantes. El decodificador, en el otro extremo, debe deshacer esas combinaciones, y para eso necesita también las operaciones inversas: la resta y la división. Por lo tanto, el alfabeto de símbolos no puede ser un conjunto cualquiera: debe estar dotado de las cuatro operaciones de la aritmética elemental, con sus reglas habituales (leyes conmutativa, asociativa y distributiva, y existencia de los elementos neutros 0 y 1). Y hay una restricción adicional que es específica de esta disciplina: el alfabeto debe ser *finito*, porque los símbolos que viajan por el canal son bits, bytes o, en general, palabras de m bits, y su cantidad es limitada.

Un conjunto finito de elementos dotado de las cuatro operaciones de la aritmética elemental se denomina *cuerpo de Galois* (*Galois field*), en honor a Évariste Galois.

Toda la maquinaria de esta unidad apunta a construir esos cuerpos, en particular los de 2^m elementos, y a aprender a calcular en ellos. Sobre esa base se apoyan los códigos de bloque lineales, los códigos cíclicos, los BCH y los Reed–Solomon: sin ella, los algoritmos de decodificación que se estudian más adelante (Berlekamp–Massey, búsqueda de Chien, Forney) no tienen dónde apoyarse.

El recorrido de la unidad es el siguiente: primero se define la estructura más simple, el *grupo*, que aparece cuando se considera una sola operación; luego el *cuerpo*, que resulta de combinar dos operaciones compatibles entre sí; a continuación se estudia la aritmética de polinomios sobre el cuerpo binario, que es la herramienta con la que se construyen los cuerpos $GF(2^m)$; y se cierra con un repaso de espacios vectoriales y matrices sobre cuerpos finitos, que es el lenguaje en el que se escribirán los códigos de bloque lineales de la unidad siguiente.

2. Grupos

2.1. Definición

Sea G un conjunto de elementos y sea $*$ una operación binaria definida sobre G , es decir, una regla que a cada par ordenado de elementos a y b de G le asigna un elemento $a * b$. El conjunto G es *cerrado* bajo la operación $*$ si $a * b$ pertenece a G para todo par de elementos de G .

El conjunto G , junto con la operación $*$, forma un *grupo* si se cumplen las siguientes condiciones:

1. La operación $*$ es **asociativa**: para todo a, b y c en G vale $a * (b * c) = (a * b) * c$.
2. G contiene un **elemento neutro** (o identidad) e tal que, para todo a en G , $a * e = e * a = a$.
3. Para todo elemento a de G existe un **elemento inverso** a' en G tal que $a * a' = a' * a = e$.

Si además la operación es **conmutativa**, esto es, si $a * b = b * a$ para todo par de elementos, el grupo se denomina *conmutativo* o *abeliano*. La cantidad de elementos de un grupo es su *orden*; un grupo con una cantidad finita de elementos es un *grupo finito*.

De la definición se deducen dos hechos que se usan de manera implícita todo el tiempo: el elemento neutro de un grupo es **único**, y el inverso de cada elemento es **único**. La demostración es breve y vale la pena hacerla en clase, porque muestra el estilo de razonamiento algebraico. Si e y e' fueran dos neutros, entonces $e = e * e' = e'$. Si a' y a'' fueran dos inversos de a , entonces

$$a' = a' * e = a' * (a * a'') = (a' * a) * a'' = e * a'' = a''.$$

2.2. Dos grupos finitos fundamentales

Los dos ejemplos siguientes no son decorativos: son exactamente los dos grupos que se esconden dentro de todo cuerpo finito, uno bajo la suma y otro bajo la multiplicación.

Suma módulo m . Sea m un entero positivo y sea $G = \{0, 1, 2, \dots, m-1\}$. Se define la suma módulo m , que se denota $\oplus$, como sigue: para i y j en G ,

$$i \oplus j = r,$$

donde r es el resto de dividir $i + j$ por m . Como el resto es un entero entre 0 y $m-1$, el conjunto es cerrado bajo la operación. Es sencillo verificar que $\oplus$ es conmutativa y asociativa, que 0 es el elemento neutro, y que el inverso de i es $m-i$ (mientras que el inverso de 0 es 0). Por lo tanto, G con la suma módulo m es un grupo conmutativo de orden m , que se denomina *grupo aditivo*. El caso $m = 2$ es el que más interesa: el conjunto $\{0, 1\}$ con la suma módulo 2, que no es otra cosa que la operación OR-exclusiva (XOR).

Multiplicación módulo p . Sea p un número *primo* y sea $G = \{1, 2, \dots, p-1\}$. Se define la multiplicación módulo p , que se denota $\odot$, como sigue: para i y j en G ,

$$i \odot j = r,$$

donde r es el resto de dividir el producto entero $i \cdot j$ por p . Aquí el requisito de que p sea primo es esencial. Como p es primo y tanto i como j son menores que p , el producto $i \cdot j$ no es divisible por p , de modo que el resto r es distinto de cero y pertenece a G : el conjunto es cerrado. Puede demostrarse que la operación es conmutativa y asociativa, que 1 es el elemento neutro, y que todo elemento tiene inverso multiplicativo dentro de G . Se obtiene así un grupo conmutativo de orden $p-1$, denominado *grupo multiplicativo*.

Conviene detenerse en el papel del carácter primo de p . Si p no fuera primo, la construcción falla. Por ejemplo, con $m = 4$ se tendría $2 \odot 2 = 0$: el producto de dos elementos no nulos daría cero, y en consecuencia 2 no tendría inverso multiplicativo. Este detalle, que parece menor, es la razón profunda por la cual los cuerpos finitos sólo existen cuando la cantidad de elementos es una potencia de un primo, y es lo que obliga a construir $GF(2^m)$ con la maquinaria de polinomios que se desarrolla más adelante, en lugar de usar simplemente aritmética módulo 2^m .

3. Cuerpos

3.1. Definición

Un grupo tiene una sola operación. Un cuerpo tiene dos, y la clave está en cómo se relacionan entre sí. Sea F un conjunto de elementos sobre el que se definen dos operaciones binarias, la *suma* (+) y la *multiplicación* ($\cdot$). El conjunto F , junto con esas dos operaciones, es un *cuerpo* (*field*) si se cumplen las siguientes tres condiciones:

1. F es un **grupo conmutativo bajo la suma**. El elemento neutro de la suma se denomina *cero* y se denota 0.
2. Los elementos **no nulos** de F forman un **grupo conmutativo bajo la multiplicación**. El elemento neutro de la multiplicación se denomina *unidad* y se denota 1.
3. La multiplicación es **distributiva** respecto de la suma: para todo a, b y c en F vale $a \cdot (b + c) = a \cdot b + a \cdot c$.

La cantidad de elementos del cuerpo es su *orden*. Un cuerpo con una cantidad finita de elementos es un *cuerpo finito* o cuerpo de Galois. El inverso aditivo de a se denota $-a$, de modo que la resta se define como $a - b = a + (-b)$; el inverso multiplicativo de $a \neq 0$ se denota a^{-1} , de modo que la división se define como $a/b = a \cdot b^{-1}$ para $b \neq 0$. Con esto, las cuatro operaciones de la aritmética elemental quedan disponibles, que era exactamente lo que se buscaba.

Obsérvese la sutileza en la condición 2: el cero queda *excluido* del grupo multiplicativo. Esto es necesario, porque el cero no puede tener inverso multiplicativo. Un cuerpo de orden q tiene entonces un grupo aditivo de orden q y un grupo multiplicativo de orden $q - 1$.

3.2. Propiedades elementales

De la definición se derivan las siguientes propiedades, válidas en cualquier cuerpo:

1. Para todo a en F , $a \cdot 0 = 0 \cdot a = 0$.
2. Si $a \neq 0$ y $b \neq 0$, entonces $a \cdot b \neq 0$. Dicho de otro modo, en un cuerpo no existen *divisores de cero*.
3. Como consecuencia de lo anterior, si $a \cdot b = 0$ y $a \neq 0$, entonces necesariamente $b = 0$.
4. Para todo a y b en F , $-(a \cdot b) = (-a) \cdot b = a \cdot (-b)$.
5. **Ley de cancelación:** para $a \neq 0$, si $a \cdot b = a \cdot c$, entonces $b = c$.

La ley de cancelación tiene una consecuencia práctica que resulta útil al construir las tablas de un cuerpo a mano: en cada fila (y en cada columna) de la tabla de multiplicación, descontando la fila y la columna del cero, todos los elementos deben ser *distintos*. Lo mismo vale, por la misma razón, para las filas y columnas de la tabla de suma. En efecto, si en la tabla de suma se tuviera $a + b = a + c$ con $b \neq c$, sumando $-a$ a ambos lados y aplicando la asociatividad se llegaría a $b = c$, lo que es una contradicción. Esta observación permite completar las tablas de cuerpos pequeños por eliminación, casi como un sudoku.

3.3. El cuerpo binario $GF(2)$

El cuerpo más importante para este curso es también el más simple. El conjunto $\{0, 1\}$ con la suma y la multiplicación módulo 2 es un cuerpo de dos elementos, que se denota $GF(2)$. Sus tablas se muestran en la Tabla 1.

Tabla 1: Tablas de suma y multiplicación en el cuerpo binario $GF(2)$.

+	0	1
0	0	1
1	1	0

·	0	1
0	0	0
1	0	1

La única entrada que exige justificación es $1 + 1$. El elemento 1 debe tener un inverso aditivo, es decir, debe existir en su fila un elemento que sumado a 1 produzca

0. Como el único candidato disponible es el propio 1, resulta forzosamente $1 + 1 = 0$. De aquí se sigue una propiedad que se usará sin cesar a lo largo del curso: en $GF(2)$, y en general en todo cuerpo de característica 2, **cada elemento es su propio inverso aditivo**, de modo que sumar y restar son la misma operación, y $a + a = 0$ para todo a .

La importancia práctica de $GF(2)$ es difícil de exagerar: los circuitos que implementan sus tablas de suma y multiplicación son, respectivamente, la compuerta XOR y la compuerta AND. Toda la aritmética de los códigos binarios se construye con esas dos compuertas.

3.4. Cuerpos primos y la restricción sobre el orden

El mismo razonamiento que en $GF(2)$ se generaliza: para cualquier número primo p , el conjunto $\{0, 1, \dots, p-1\}$ con la suma y la multiplicación módulo p es un cuerpo de p elementos, denominado *cuerpo primo* y denotado $GF(p)$. La Tabla 2 muestra el caso $p = 3$: la aritmética resulta ser, simplemente, la aritmética módulo 3.

Tabla 2: Tablas de suma y multiplicación en $GF(3)$: la aritmética es módulo 3.

+	0	1	2
0	0	1	2
1	1	2	0
2	2	0	1

·	0	1	2
0	0	0	0
1	0	1	2
2	0	2	1

Ahora bien, no existe un cuerpo finito de cualquier cantidad de elementos. Un resultado central de la teoría establece que **existe un cuerpo finito de q elementos si y sólo si q es la potencia de un número primo**, esto es, $q = p^m$ con p primo y m entero positivo. Además, ese cuerpo es esencialmente único: dos cuerpos con la misma cantidad de elementos son isomorfos, lo que significa que existe una correspondencia biunívoca entre ellos que preserva tanto la suma como la multiplicación. En consecuencia, todo lo que se calcule en una “versión” del cuerpo se traslada mecánicamente a cualquier otra, y la elección de una construcción concreta responde únicamente a razones de conveniencia computacional.

En el caso $m = 1$ se recuperan los cuerpos primos $GF(p)$, cuya aritmética es la aritmética módulo p . Para $m > 1$, en cambio, la aritmética **no** es la aritmética módulo q ; ya se vio, con el contraejemplo de $2 \odot 2 = 0$ módulo 4, por qué la vía directa fracasa. La construcción correcta de $GF(p^m)$ es el objeto de las secciones que siguen, restringida al caso $p = 2$, que es el que interesa en ingeniería electrónica, donde los símbolos son grupos de m bits. El cuerpo $GF(2)$ se denomina, en ese contexto, *cuerpo base* (*ground field*) de $GF(2^m)$.

3.5. Característica de un cuerpo

Sea $GF(q)$ un cuerpo finito. Considérese la sucesión de sumas del elemento unidad consigo mismo:

$$\sum_{i=1}^1 1, \quad \sum_{i=1}^2 1 = 1 + 1, \quad \sum_{i=1}^3 1 = 1 + 1 + 1, \quad \dots$$

Como el cuerpo es cerrado bajo la suma, todas estas sumas son elementos del cuerpo; y como el cuerpo tiene una cantidad finita de elementos, no pueden ser todas distintas: en algún momento la sucesión se repite. De ahí se deduce que existe un menor entero positivo λ tal que

$$\sum_{i=1}^{\lambda} 1 = 0.$$

Ese entero λ se denomina *característica* del cuerpo. Puede demostrarse que **la característica de un cuerpo finito es siempre un número primo**. La característica de $GF(2)$ es 2, porque $1 + 1 = 0$; y, en general, la característica de $GF(2^m)$ también es 2. Ésta es la razón formal de que en todos los cuerpos con los que se trabaja en el curso valga $a + a = 0$, y de que sumar y restar sean la misma operación.

4. Aritmética de polinomios sobre $GF(2)$

Para construir cuerpos de 2^m elementos se necesita una herramienta intermedia: los polinomios con coeficientes en $GF(2)$. La estrategia es análoga a la que se emplea para construir los números complejos a partir de los reales. Un número complejo se manipula como un polinomio en el símbolo j , y el producto de dos complejos se obtiene multiplicando los polinomios y reemplazando luego j^2 por -1 ; equivalentemente, tomando el resto de la división por el polinomio $D^2 + 1$, que no tiene raíces reales. La misma idea, aplicada sobre $GF(2)$ en lugar de sobre los reales, produce los cuerpos $GF(2^m)$.

4.1. Polinomios sobre $GF(2)$ y sus operaciones

Un *polinomio sobre $GF(2)$* de grado n es una expresión de la forma

$$f(X) = f_0 + f_1X + f_2X^2 + \cdots + f_nX^n,$$

donde cada coeficiente f_i pertenece a $\{0, 1\}$ y $f_n \neq 0$. Existen exactamente 2^n polinomios de grado n sobre $GF(2)$. Los polinomios se suman, se multiplican y se dividen de la manera habitual, con la única salvedad de que las operaciones sobre los coeficientes se realizan en $GF(2)$, es decir, módulo 2.

La suma se efectúa coeficiente a coeficiente. Por ejemplo, con $a(X) = 1 + X + X^3 + X^5$ y $b(X) = 1 + X^2 + X^3 + X^4 + X^7$ se obtiene

$$a(X) + b(X) = X + X^2 + X^4 + X^5 + X^7,$$

donde el término independiente y el término en X^3 se cancelan porque $1 + 1 = 0$.

La multiplicación es la convolución usual de los coeficientes, con sumas módulo 2. Y la división también funciona como es habitual: dados $f(X)$ y $g(X) \neq 0$, existen dos polinomios únicos $q(X)$ (el cociente) y $r(X)$ (el resto) tales que

$$f(X) = q(X)g(X) + r(X),$$

con el grado de $r(X)$ estrictamente menor que el grado de $g(X)$. Este algoritmo de división es la operación que se implementará más adelante en hardware, con registros de desplazamiento realimentados, tanto para codificar códigos cíclicos como para calcular síndromes.

4.2. Raíces y factores

Se dice que un elemento a es una *raíz* del polinomio $f(X)$ si $f(a) = 0$. Vale el mismo resultado que en el álgebra ordinaria: **si a es raíz de $f(X)$, entonces $f(X)$ es divisible por $X + a$** , y recíprocamente. (Sobre $GF(2)$ se escribe $X + a$ en lugar de $X - a$, ya que la resta coincide con la suma.)

Por ejemplo, el polinomio $f(X) = 1 + X^2 + X^3 + X^4$ tiene a 1 como raíz, puesto que $f(1) = 1 + 1 + 1 + 1 = 0$. Efectivamente, $f(X)$ resulta divisible por $X + 1$.

4.3. Polinomios irreducibles

Un polinomio $p(X)$ sobre $GF(2)$, de grado m , es *irreducible sobre $GF(2)$* si no admite ningún divisor de grado mayor que cero y menor que m ; es decir, si no puede factorizarse como producto de dos polinomios de grado menor, ambos con coeficientes en $GF(2)$.

Un ejemplo: $X^3 + X + 1$ es irreducible. Se verifica sin dificultad, ya que un polinomio de grado 3 sólo podría factorizarse teniendo un factor de grado 1, esto es, una raíz en $GF(2)$; pero $f(0) = 1$ y $f(1) = 1$, de modo que ni 0 ni 1 son raíces, y por lo tanto es irreducible. De los ocho polinomios de grado 3 sobre $GF(2)$, sólo dos resultan irreducibles: $X^3 + X + 1$ y $X^3 + X^2 + 1$. Para grado 4 hay exactamente tres: $X^4 + X + 1$, $X^4 + X^3 + 1$ y $X^4 + X^3 + X^2 + X + 1$.

Nótese que el criterio de la raíz sólo alcanza para grados 2 y 3. Para grado 4 o mayor, un polinomio puede no tener raíces en $GF(2)$ y ser, sin embargo, reducible (por ejemplo, si factoriza como producto de dos irreducibles de grado 2).

Un resultado importante, que se retomará al construir el cuerpo, establece que **todo polinomio irreducible de grado m sobre $GF(2)$ divide a $X^{2^m-1} + 1$** .

4.4. Polinomios primitivos

Entre los polinomios irreducibles interesa una subfamilia especial. Un polinomio irreducible $p(X)$ de grado m es *primitivo* si el menor entero positivo n para el cual $p(X)$ divide a $X^n + 1$ es precisamente $n = 2^m - 1$. Ese entero n se denomina *período* del polinomio, de manera que un polinomio irreducible es primitivo cuando su período alcanza el valor máximo posible.

El ejemplo canónico de la diferencia entre ambos conceptos aparece en grado 4. Los tres polinomios irreducibles de grado 4 se enumeraron más arriba, pero sólo dos de ellos son primitivos: $X^4 + X + 1$ y $X^4 + X^3 + 1$. El tercero, $X^4 + X^3 + X^2 + X + 1$, es irreducible pero **no** primitivo, porque divide a $X^5 + 1$ (su período es 5, y no 15). Los tres sirven para construir un cuerpo de 16 elementos, y los tres cuerpos resultantes son isomorfos entre sí; pero, como se verá enseguida, la elección de un polinomio primitivo hace que la construcción y los circuitos asociados sean mucho más simples. Por esa razón, en control de errores se utilizan exclusivamente polinomios primitivos.

Los polinomios primitivos no son fáciles de encontrar a mano, pero están tabulados. Para cada grado m existe al menos uno, y los libros de referencia incluyen tablas con el polinomio primitivo de menor cantidad de términos para cada m . Dos que aparecerán con frecuencia en el curso son $X^3 + X + 1$ (para $GF(2^3)$) y $X^8 + X^4 + X^3 + X^2 + 1$ (para $GF(2^8)$, el cuerpo sobre el que trabajan los códigos BCH y Reed–Solomon del sistema 400ZR del trabajo práctico final).

4.5. Una propiedad peculiar de la característica 2

Vale la pena registrar una identidad que es falsa en el álgebra ordinaria pero verdadera aquí, y que se usará varias veces. Para cualquier polinomio $f(X)$ sobre $GF(2)$,

$$(f(X))^2 = f(X^2) \quad (1)$$

La razón es que, al elevar al cuadrado, todos los productos cruzados aparecen *dos veces* y por lo tanto se cancelan, ya que $a + a = 0$. Sólo sobreviven los cuadrados de cada término. Por iteración se obtiene también $(f(X))^{2^\ell} = f(X^{2^\ell})$. Esta identidad, aparentemente anecdótica, es la que da origen a los *conjugados* y a los polinomios mínimos, que constituyen la base de la construcción de los códigos BCH.

5. Construcción del cuerpo de Galois $GF(2^m)$

5.1. La idea de la construcción

Se parte de los dos elementos 0 y 1 de $GF(2)$ y se introduce un símbolo nuevo, α . Se define una multiplicación que genera las potencias sucesivas de α :

$$F = \{0, 1, \alpha, \alpha^2, \alpha^3, \dots\},$$

con las reglas naturales $0 \cdot \alpha^j = 0$, $1 \cdot \alpha^j = \alpha^j$ y $\alpha^i \cdot \alpha^j = \alpha^{i+j}$. Tal como está, este conjunto es infinito. Para que contenga exactamente 2^m elementos y sea cerrado, hay que imponer una condición sobre α , y ésta es precisamente la función del polinomio primitivo.

Sea $p(X)$ un polinomio primitivo de grado m sobre $GF(2)$, y **postúlese que α es una raíz suya**, es decir, $p(\alpha) = 0$. Esta es la maniobra clave, y es la misma que se hace al introducir la unidad imaginaria: j se define como una raíz de $D^2 + 1$, un polinomio que no tiene raíces reales. De manera análoga, α no pertenece a $GF(2)$, sino a una extensión suya.

Como $p(X)$ es primitivo de grado m , divide a $X^{2^m-1} + 1$, de modo que puede escribirse $X^{2^m-1} + 1 = q(X)p(X)$. Reemplazando X por α y usando que $p(\alpha) = 0$ se obtiene

$$\alpha^{2^m-1} + 1 = q(\alpha) \cdot 0 = 0, \quad \text{es decir} \quad \alpha^{2^m-1} = 1.$$

Las potencias de α se cierran entonces sobre sí mismas con período $2^m - 1$. El conjunto

$$F^* = \{0, 1, \alpha, \alpha^2, \dots, \alpha^{2^m-2}\}$$

tiene exactamente 2^m elementos y es cerrado bajo la multiplicación. Puede demostrarse que, con la suma que se define a continuación, F^* es un cuerpo: es el cuerpo de Galois $GF(2^m)$.

5.2. Las tres representaciones de los elementos

Para sumar hace falta otra representación. Como α satisface $p(\alpha) = 0$, la relación $p(\alpha) = 0$ permite expresar α^m en función de las potencias menores, y por sustitución

reiterada toda potencia α^i se escribe como un polinomio en α de grado a lo sumo $m - 1$:

$$\alpha^i = a_{i,0} + a_{i,1}\alpha + a_{i,2}\alpha^2 + \cdots + a_{i,m-1}\alpha^{m-1},$$

con coeficientes en $GF(2)$. Puede demostrarse que estos $2^m - 1$ polinomios son todos distintos, de modo que la correspondencia es biunívoca. Y como un polinomio de grado $m - 1$ queda determinado por sus m coeficientes, cada elemento admite además una tercera representación como m -tupla de bits:

$$\alpha^i \longleftrightarrow (a_{i,0}, a_{i,1}, \dots, a_{i,m-1}).$$

Se dispone entonces de tres representaciones equivalentes del mismo objeto, y cada una es cómoda para una tarea distinta:

- La representación como **potencia** de α es la conveniente para **multiplicar** y dividir: multiplicar es sumar exponentes módulo $2^m - 1$.
- Las representaciones **polinómica** y de m -**tupla** son las convenientes para **sumar**: sumar es sumar los coeficientes módulo 2, es decir, hacer el XOR bit a bit de las m -tuplas.

Saber pasar de una representación a la otra con soltura es la habilidad práctica central de esta unidad.

5.3. Ejemplo: construcción de $GF(2^4)$

Sea $m = 4$ y tómesese el polinomio primitivo $p(X) = 1 + X + X^4$. Postulando $p(\alpha) = 1 + \alpha + \alpha^4 = 0$ se obtiene la relación fundamental

$$\alpha^4 = 1 + \alpha,$$

que se aplica reiteradamente para generar el resto de los elementos. Por ejemplo:

$$\begin{aligned}\alpha^5 &= \alpha \cdot \alpha^4 = \alpha(1 + \alpha) = \alpha + \alpha^2, \\ \alpha^6 &= \alpha \cdot \alpha^5 = \alpha(\alpha + \alpha^2) = \alpha^2 + \alpha^3, \\ \alpha^7 &= \alpha \cdot \alpha^6 = \alpha(\alpha^2 + \alpha^3) = \alpha^3 + \alpha^4 = \alpha^3 + 1 + \alpha = 1 + \alpha + \alpha^3.\end{aligned}$$

Continuando de este modo se obtienen los 15 elementos no nulos, y en $\alpha^{15} = 1$ el ciclo se cierra. El resultado completo se muestra en la Tabla 3.

Tabla 3: Las tres representaciones de los elementos de $GF(2^4)$ generado por el polinomio primitivo $p(X) = 1+X+X^4$. Adaptado de Lin & Costello, Tabla 2.8.

Potencia	Polinomio	4-tupla
0	0	(0 0 0 0)
1	1	(1 0 0 0)
α	α	(0 1 0 0)
α^2	α^2	(0 0 1 0)
α^3	α^3	(0 0 0 1)
α^4	$1 + \alpha$	(1 1 0 0)
α^5	$\alpha + \alpha^2$	(0 1 1 0)
α^6	$\alpha^2 + \alpha^3$	(0 0 1 1)
α^7	$1 + \alpha + \alpha^3$	(1 1 0 1)
α^8	$1 + \alpha^2$	(1 0 1 0)
α^9	$\alpha + \alpha^3$	(0 1 0 1)
α^{10}	$1 + \alpha + \alpha^2$	(1 1 1 0)
α^{11}	$\alpha + \alpha^2 + \alpha^3$	(0 1 1 1)
α^{12}	$1 + \alpha + \alpha^2 + \alpha^3$	(1 1 1 1)
α^{13}	$1 + \alpha^2 + \alpha^3$	(1 0 1 1)
α^{14}	$1 + \alpha^3$	(1 0 0 1)

Con la tabla a la vista, la aritmética se vuelve mecánica. Para **multiplicar** basta sumar exponentes módulo 15:

$$\alpha^5 \cdot \alpha^7 = \alpha^{12}, \quad \alpha^{12} \cdot \alpha^7 = \alpha^{19} = \alpha^4.$$

Para **dividir** se multiplica por el inverso, y el inverso de α^i es simplemente α^{15-i} :

$$\frac{\alpha^4}{\alpha^{12}} = \alpha^4 \cdot \alpha^3 = \alpha^7.$$

Para **sumar** se pasa a la representación polinómica (o, equivalentemente, se hace el XOR de las 4-tuplas):

$$\alpha^5 + \alpha^7 = (\alpha + \alpha^2) + (1 + \alpha + \alpha^3) = 1 + \alpha^2 + \alpha^3 = \alpha^{13}.$$

Un caso que conviene mostrar en clase por lo contraintuitivo que resulta a primera vista:

$$1 + \alpha^5 + \alpha^{10} = 1 + (\alpha + \alpha^2) + (1 + \alpha + \alpha^2) = 0.$$

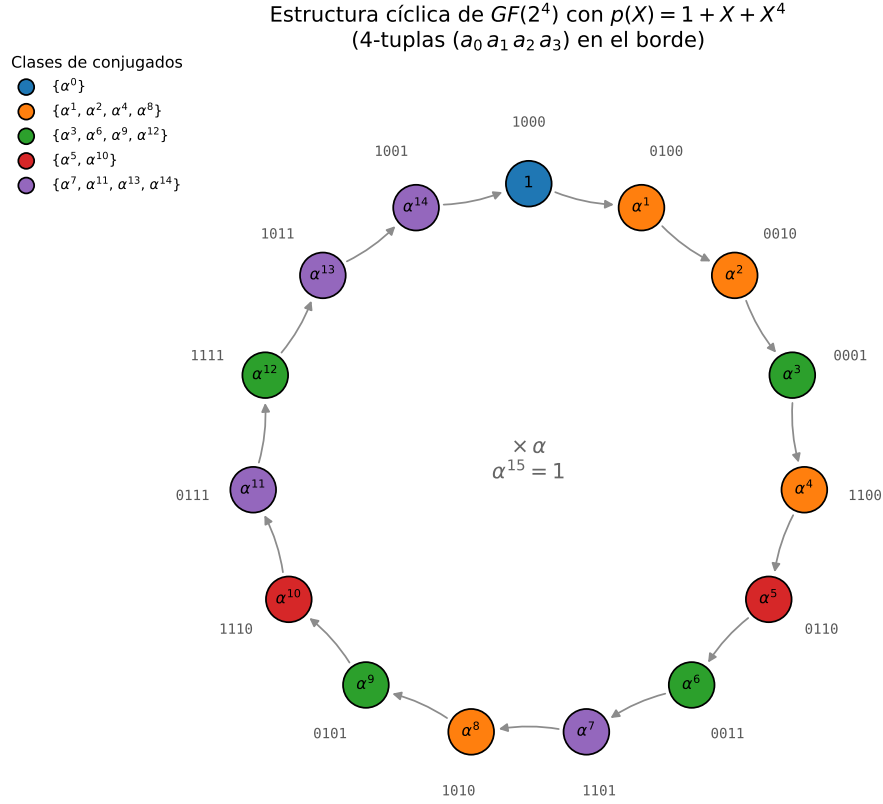


Figura 1: Los 15 elementos no nulos de $GF(2^4)$ dispuestos como potencias sucesivas del elemento primitivo α . Multiplicar por α equivale a avanzar una posición sobre el círculo; multiplicar por α^i , a avanzar i posiciones. Los colores agrupan las clases de conjugados (Sección 6.2). Figura generada con el script `gf16_circulo.py`.

La figura anterior resume la estructura del cuerpo mejor que cualquier tabla: el grupo multiplicativo de $GF(2^m)$ es *cíclico*, y α es el elemento que lo recorre entero.

6. Propiedades básicas de $GF(2^m)$

6.1. Orden de un elemento y elementos primitivos

Sea β un elemento no nulo de $GF(2^m)$. Como el grupo multiplicativo es finito, las potencias sucesivas de β deben repetirse; se define el *orden* de β como el menor entero positivo n tal que $\beta^n = 1$. Valen dos resultados fundamentales:

1. Todo elemento no nulo β de $GF(2^m)$ satisface $\beta^{2^m-1} = 1$.
2. El orden n de todo elemento no nulo **divide** a $2^m - 1$.

Un elemento cuyo orden es exactamente $2^m - 1$ se denomina *elemento primitivo*: sus potencias generan todos los elementos no nulos del cuerpo. Por construcción, el α

raíz del polinomio primitivo es un elemento primitivo. Pero no es el único: en $GF(2^4)$, por ejemplo, hay ocho elementos primitivos, mientras que los demás tienen órdenes 3 o 5, que son justamente los divisores de 15.

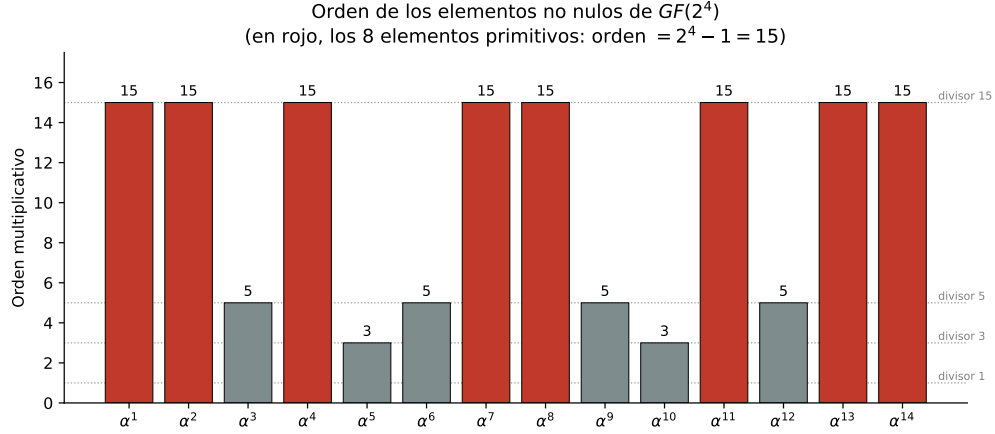


Figura 2: Orden de cada elemento no nulo de $GF(2^4)$. Todos los órdenes son divisores de 15, y los elementos de orden máximo (15) son los elementos primitivos del cuerpo. Figura generada con el script `gf16_ordenes.py`.

6.2. Conjugados

Aquí es donde la identidad (1) rinde sus frutos. Sea $f(X)$ un polinomio con coeficientes en $GF(2)$ y sea β un elemento de $GF(2^m)$ que es raíz de $f(X)$, es decir, $f(\beta) = 0$. Entonces, elevando al cuadrado,

$$(f(\beta))^2 = f(\beta^2) = 0,$$

de modo que β^2 **también es raíz de $f(X)$** . Repitiendo el argumento, también lo son β^4 , β^8 , y en general β^{2^ℓ} para todo ℓ . Estos elementos se denominan *conjugados* de β .

El conjunto de los conjugados de un elemento es finito, porque el cuerpo lo es: existe un menor entero e tal que $\beta^{2^e} = \beta$, y los conjugados distintos de β son entonces

$$\beta, \beta^2, \beta^{2^2}, \dots, \beta^{2^{e-1}}.$$

Ese conjunto se denomina *clase de conjugados* (o *coset ciclotómico*), y puede demostrarse que e divide a m . En $GF(2^4)$, por ejemplo, las clases de conjugados son las siguientes:

$$\{1\}, \quad \{\alpha, \alpha^2, \alpha^4, \alpha^8\}, \quad \{\alpha^3, \alpha^6, \alpha^{12}, \alpha^9\}, \quad \{\alpha^5, \alpha^{10}\}, \quad \{\alpha^7, \alpha^{14}, \alpha^{13}, \alpha^{11}\}.$$

Estas clases son las que se colorean en la Figura 1. Obsérvese que las clases *particionan* el conjunto de elementos no nulos, y que el tamaño de cada una (1, 4, 4, 2, 4) divide a $m = 4$.

Un resultado que se usará al construir los códigos BCH: si β es un elemento primitivo, todos sus conjugados también lo son.

6.3. Polinomios mínimos

Todo elemento β de $GF(2^m)$ es raíz de algún polinomio con coeficientes en $GF(2)$ (por ejemplo, de $X^{2^m} + X$). Entre todos ellos, hay uno de grado mínimo, y se demuestra que ese polinomio es **único** y **irreducible**: se lo denomina *polinomio mínimo* de β y se lo denota $\phi(X)$.

La conexión con los conjugados es directa y elegante. Como todos los conjugados de β son necesariamente raíces de $\phi(X)$, y como puede demostrarse que $\phi(X)$ no tiene ninguna otra raíz, resulta que el polinomio mínimo de β es exactamente el producto de los factores asociados a su clase de conjugados:

$$\phi(X) = \prod_{i=0}^{e-1} (X + \beta^{2^i}) \quad (2)$$

donde e es el tamaño de la clase de conjugados de β . En consecuencia, **el grado del polinomio mínimo de cualquier elemento de $GF(2^m)$ divide a m** . Aunque el producto se realiza en $GF(2^m)$, el resultado tiene todos sus coeficientes en $GF(2)$: los términos cruzados se cancelan sistemáticamente.

La Tabla 4 reúne todos los polinomios mínimos de $GF(2^4)$. Nótese la correspondencia uno a uno con las clases de conjugados listadas más arriba.

Tabla 4: Polinomios mínimos de los elementos de $GF(2^4)$ generado por $p(X) = 1 + X + X^4$. Adaptado de Lin & Costello, Tabla 2.9.

Clase de conjugados	Polinomio mínimo
0	X
1	$1 + X$
$\alpha, \alpha^2, \alpha^4, \alpha^8$	$1 + X + X^4$
$\alpha^3, \alpha^6, \alpha^9, \alpha^{12}$	$1 + X + X^2 + X^3 + X^4$
α^5, α^{10}	$1 + X + X^2$
$\alpha^7, \alpha^{11}, \alpha^{13}, \alpha^{14}$	$1 + X^3 + X^4$

Vale la pena verificar un caso a mano en clase, porque el procedimiento aparecerá una y otra vez. Tómese $\beta = \alpha^7$. Sus conjugados son $\beta^2 = \alpha^{14}$, $\beta^{2^2} = \alpha^{28} = \alpha^{13}$ y $\beta^{2^3} = \alpha^{56} = \alpha^{11}$; en el paso siguiente se vuelve a β , de modo que la clase tiene cuatro elementos y $\phi(X)$ tiene grado 4. Escribiendo $\phi(X) = a_0 + a_1X + a_2X^2 + a_3X^3 + X^4$, imponiendo $\phi(\alpha^7) = 0$, reemplazando cada potencia por su representación polinómica de la Tabla 3 y agrupando los coeficientes de 1, α , α^2 y α^3 , se obtiene un sistema lineal sobre $GF(2)$ cuya solución es $a_0 = 1$, $a_1 = a_2 = 0$, $a_3 = 1$. Es decir, $\phi(X) = 1 + X^3 + X^4$, en coincidencia con la tabla.

6.4. La factorización de $X^{2^m-1} + 1$

Los resultados anteriores se sintetizan en una identidad que es la puerta de entrada a los códigos cíclicos y BCH. Los $2^m - 1$ elementos no nulos de $GF(2^m)$ son exactamente las $2^m - 1$ raíces del polinomio $X^{2^m-1} + 1$. Agrupando esas raíces por clases de conjugados, y recordando que a cada clase le corresponde un polinomio mínimo, se concluye que

$$X^{2^m-1} + 1 = \prod_i \phi_i(X) \quad (3)$$

donde el producto recorre los polinomios mínimos de todas las clases de conjugados de los elementos no nulos. Con los datos de la Tabla 4, para $m = 4$:

$$X^{15} + 1 = (1 + X) (1 + X + X^4) (1 + X + X^2 + X^3 + X^4) (1 + X + X^2) (1 + X^3 + X^4).$$

Esta factorización es la que permitirá, en las unidades de códigos cíclicos y BCH, *elegir* el polinomio generador de un código como el producto de aquellos polinomios mínimos cuyas raíces se quiere que anulen las palabras de código. Toda la construcción de los BCH descansa sobre esta identidad.

7. Cómputos en cuerpos de Galois

Saber que el cuerpo existe es una cosa; calcular en él de manera eficiente, en software o en hardware, es otra. Esta sección reúne las técnicas prácticas, que reaparecen literalmente en los módulos del trabajo práctico final (sumador, multiplicador y exponenciador sobre $GF(2^8)$).

7.1. Tablas de logaritmo y antilogaritmo

La técnica más difundida en software consiste en precomputar dos tablas de 2^m entradas:

- La tabla de **antilogaritmo** (o de exponenciación), que dado el exponente i devuelve la m -tupla correspondiente a α^i .
- La tabla de **logaritmo**, que dada la m -tupla devuelve el exponente i .

Con ambas tablas, las cuatro operaciones se resuelven de manera inmediata: la suma es un XOR directo de las m -tuplas; el producto de dos elementos no nulos se calcula pasando a logaritmos, sumando módulo $2^m - 1$ y volviendo con el antilogaritmo; la división es igual pero restando; y el inverso de α^i es α^{2^m-1-i} . Es imprescindible tratar el 0 como caso especial, porque no tiene logaritmo. Para $GF(2^8)$, que es el cuerpo del sistema 400ZR, las dos tablas ocupan 256 bytes cada una, lo que las vuelve una solución más que razonable en software.

7.2. Logaritmos de Zech

Existe una alternativa que permite operar *siempre* en el dominio de los exponentes, incluso para sumar. Se define el *logaritmo de Zech* $z(i)$ mediante la relación

$$1 + \alpha^i = \alpha^{z(i)}.$$

Con esa tabla precomputada, la suma de dos elementos cualesquiera se reduce a una suma de exponentes:

$$\alpha^i + \alpha^j = \alpha^i (1 + \alpha^{j-i}) = \alpha^{i+z(j-i)}, \quad j > i.$$

Por ejemplo, en $GF(2^4)$, con la Tabla 5: $\alpha^7 + \alpha^{12} = \alpha^7 (1 + \alpha^5) = \alpha^7 \cdot \alpha^{10} = \alpha^{17} = \alpha^2$.

Tabla 5: Logaritmos de Zech en $GF(2^4)$ con $p(X) = 1 + X + X^4$:
el valor $z(i)$ satisface $1 + \alpha^i = \alpha^{z(i)}$.

i	1	2	3	4	5	6	7	8	9	10	11	12	13	14
$z(i)$	4	8	14	1	10	13	9	2	7	5	12	11	6	3

La tabla de Zech tiene $2^m - 1$ entradas, contra las dos tablas de 2^m del método anterior. Su ventaja es que evita las conversiones de ida y vuelta entre representaciones; su desventaja, que hay que tratar con cuidado el caso $i = j$ (donde la suma da 0) y el caso $\alpha^i = 1$.

7.3. Implementación en hardware

En hardware, las decisiones son distintas, porque las tablas grandes cuestan memoria y las compuertas son baratas.

El **sumador** sobre $GF(2^m)$ es trivial: m compuertas XOR en paralelo, una por bit, sin acarreo alguno. Ésta es una de las razones por las cuales los cuerpos de característica 2 son los preferidos en electrónica digital.

El **multiplicador** es más interesante. Multiplicar dos elementos equivale a multiplicar sus polinomios y luego reducir el resultado módulo $p(X)$. El caso particular más importante es la multiplicación *por* α , que en la representación de m -tupla consiste en un desplazamiento de un lugar seguido de una realimentación de los bits determinada por los coeficientes de $p(X)$. El circuito que lo implementa es un registro de desplazamiento realimentado linealmente (*Linear Feedback Shift Register*, LFSR), y es el ladrillo básico de casi todos los circuitos de codificación y decodificación que se verán más adelante: el codificador cíclico, el calculador de síndrome y la búsqueda de Chien se construyen todos a partir de él.

Multiplicador por α en $GF(2^4)$, $p(X) = 1 + X + X^4$

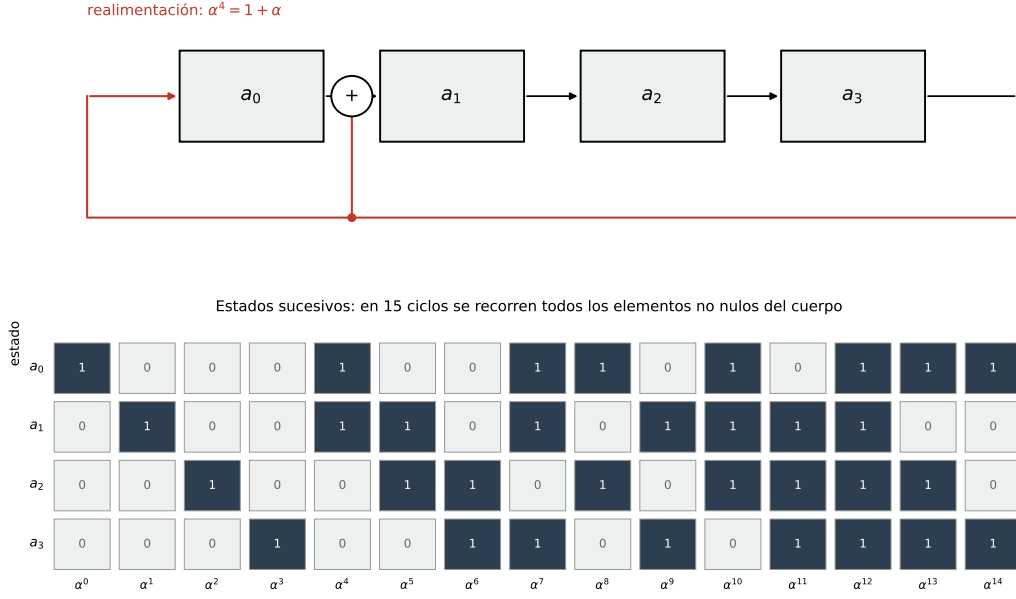


Figura 3: Registro de desplazamiento realimentado que implementa la multiplicación por α en $GF(2^4)$ con $p(X) = 1 + X + X^4$. Partiendo del estado (1000), los sucesivos ciclos de reloj recorren todos los elementos no nulos del cuerpo en el orden $\alpha^0, \alpha^1, \dots, \alpha^{14}$.

Figura generada con el script `gf16_lfsr.py`.

El multiplicador general (entre dos elementos arbitrarios) se construye combinando m etapas de este tipo con compuertas AND que seleccionan los términos según los bits del segundo operando. Esta estructura es exactamente la que se pide implementar en el módulo “aritmética sobre $GF(2^8)$ ” del trabajo práctico final.

7.4. Resolución de ecuaciones y búsqueda de raíces

Como $GF(2^m)$ es un cuerpo, todas las técnicas del álgebra ordinaria valen sobre él: se pueden resolver sistemas de ecuaciones lineales por eliminación, calcular determinantes, e invertir matrices. La única precaución es recordar que la aritmética de los coeficientes es la del cuerpo, no la de los reales, y que la resta coincide con la suma.

Un procedimiento que conviene destacar, porque anticipa directamente un algoritmo del curso, es la **búsqueda de raíces** de un polinomio con coeficientes en $GF(2^m)$. A diferencia del caso real, aquí no hay fórmulas generales, pero tampoco hacen falta: el cuerpo es finito, de modo que basta con *evaluar el polinomio en todos sus elementos* y quedarse con aquellos que lo anulan. Este método por fuerza bruta, que sería absurdo sobre los reales, es perfectamente razonable sobre un cuerpo de 256 elementos, y es exactamente la idea de la *búsqueda de Chien* que se estudiará en la unidad de códigos BCH: allí las raíces del polinomio localizador de errores indican las posiciones erróneas de la palabra recibida.

Un ejemplo instructivo, para hacer en el pizarrón: hallar las raíces de $f(X) = X^2 + \alpha^7 X + \alpha^{12}$ sobre $GF(2^4)$. Evaluando en cada uno de los 16 elementos se encuentra que $f(\alpha^6) = 0$ y $f(\alpha^{10}) = 0$; en efecto, $\alpha^6 \cdot \alpha^{10} = \alpha^{16} = \alpha$ y $\alpha^6 + \alpha^{10} = \alpha^7$, lo que

verifica las relaciones entre coeficientes y raíces.

8. Espacios vectoriales sobre cuerpos finitos

La última pieza de la unidad es el lenguaje en el que se escribirán los códigos de bloque lineales. La estructura de cuerpo permite operar con los *símbolos*; la de espacio vectorial permite operar con las *palabras*.

8.1. Definición

Sea V un conjunto de elementos, llamados *vectores*, sobre el que se define una suma, y sea F un cuerpo, cuyos elementos se llaman *escalares*. Se define además una multiplicación entre un escalar y un vector, que produce un vector. El conjunto V es un *espacio vectorial sobre el cuerpo F* si se cumplen las siguientes condiciones:

1. V es un grupo conmutativo bajo la suma.
2. Para todo a en F y todo $\mathbf{v}$ en V , el producto $a \cdot \mathbf{v}$ pertenece a V .
3. La multiplicación por escalares es distributiva respecto de ambas sumas: $a \cdot (\mathbf{u} + \mathbf{v}) = a \cdot \mathbf{u} + a \cdot \mathbf{v}$ y $(a + b) \cdot \mathbf{v} = a \cdot \mathbf{v} + b \cdot \mathbf{v}$.
4. La multiplicación por escalares es asociativa: $(a \cdot b) \cdot \mathbf{v} = a \cdot (b \cdot \mathbf{v})$.
5. Para todo $\mathbf{v}$ en V vale $1 \cdot \mathbf{v} = \mathbf{v}$, donde 1 es la unidad de F .

8.2. El espacio de las n -tuplas sobre $GF(2)$

El espacio vectorial que interesa en este curso es el de las n -tuplas binarias. Sea

$$V_n = \{(v_0, v_1, \dots, v_{n-1}) : v_i \in GF(2)\},$$

con la suma definida componente a componente (es decir, el XOR bit a bit) y la multiplicación por los escalares 0 y 1 de $GF(2)$. Es inmediato verificar que V_n es un espacio vectorial sobre $GF(2)$. Contiene 2^n vectores, y el vector nulo es $(0, 0, \dots, 0)$. Obsérvese que, por la característica 2, todo vector es su propio inverso aditivo: $\mathbf{v} + \mathbf{v} = \mathbf{0}$.

Este es exactamente el conjunto de todas las palabras posibles de longitud n que pueden circular por el canal. Un código de bloque (n, k) será, en la unidad siguiente, un *subconjunto* muy particular de V_n : un subespacio de dimensión k .

8.3. Independencia lineal, base y dimensión

Un conjunto de vectores $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k$ es *linealmente dependiente* si existen escalares $a_1, \dots, a_k$ del cuerpo, no todos nulos, tales que

$$a_1 \mathbf{v}_1 + a_2 \mathbf{v}_2 + \dots + a_k \mathbf{v}_k = \mathbf{0}.$$

En caso contrario, el conjunto es *linealmente independiente*. Una combinación de la forma anterior, con escalares arbitrarios, se denomina *combinación lineal* de los vectores.

Un conjunto de vectores linealmente independientes cuyas combinaciones lineales generan todo el espacio es una *base*, y la cantidad de vectores de una base es la *dimensión* del espacio. El espacio V_n tiene dimensión n ; una base natural es la formada por los n vectores con un único 1:

$$\mathbf{e}_0 = (1, 0, \dots, 0), \quad \mathbf{e}_1 = (0, 1, \dots, 0), \quad \dots, \quad \mathbf{e}_{n-1} = (0, 0, \dots, 1),$$

dado que todo vector de V_n se escribe de manera única como $\mathbf{v} = v_0\mathbf{e}_0 + v_1\mathbf{e}_1 + \dots + v_{n-1}\mathbf{e}_{n-1}$.

Nótese un hecho que no tiene análogo en el caso real y que es la clave del recuento de palabras de código: si un subespacio tiene dimensión k sobre $GF(2)$, entonces contiene exactamente 2^k vectores, porque cada uno de los k coeficientes de la combinación lineal admite sólo dos valores. Ahí está el origen de la relación entre los k bits de mensaje y las 2^k palabras de código de un código (n, k) .

8.4. Subespacios

Un subconjunto S de un espacio vectorial V es un *subespacio* si es, a su vez, un espacio vectorial sobre el mismo cuerpo. En la práctica alcanza con verificar dos condiciones: que el vector nulo pertenezca a S , y que S sea cerrado bajo la suma y bajo la multiplicación por escalares.

Este es, literalmente, el concepto que define a un código de bloque lineal. Un código (n, k) lineal es un subespacio de dimensión k del espacio V_n de todas las n -tuplas. La condición de cierre bajo la suma tiene una consecuencia inmediata y muy útil: **la suma de dos palabras de código es siempre otra palabra de código**, y la palabra nula siempre pertenece al código.

8.5. Producto interno y espacio dual

Dados dos vectores $\mathbf{u} = (u_0, \dots, u_{n-1})$ y $\mathbf{v} = (v_0, \dots, v_{n-1})$ de V_n , se define su *producto interno* (o producto punto) como el escalar

$$\mathbf{u} \cdot \mathbf{v} = u_0v_0 + u_1v_1 + \dots + u_{n-1}v_{n-1},$$

donde las operaciones se realizan en $GF(2)$. Si $\mathbf{u} \cdot \mathbf{v} = 0$, se dice que los vectores son *ortogonales*.

Una advertencia importante: sobre $GF(2)$, un vector no nulo puede ser ortogonal a sí mismo. Por ejemplo, $(1, 1) \cdot (1, 1) = 1 + 1 = 0$. Esto no ocurre sobre los reales, y es la razón por la cual existen los códigos *auto-duales*, que se mencionan en la unidad de códigos de bloque lineales.

Dado un subespacio S de dimensión k de V_n , el conjunto S_d de todos los vectores de V_n que son ortogonales a todos los vectores de S es también un subespacio, denominado *espacio nulo* o *espacio dual* de S . Su dimensión es

$$\dim(S_d) = n - k.$$

Esta relación es el corazón de lo que sigue: si el código es el subespacio S generado por las filas de una matriz G , su dual está generado por las filas de una matriz H de $n - k$ filas, y la condición de que una palabra pertenezca al código se expresa como $\mathbf{v} \cdot H^T = \mathbf{0}$. Esa es exactamente la ecuación del síndrome de la unidad siguiente.

9. Matrices sobre cuerpos finitos

9.1. Definiciones y rango

Una matriz de $k \times n$ sobre $GF(2)$ es un arreglo rectangular de k filas y n columnas cuyos elementos pertenecen a $GF(2)$. Sus filas pueden verse como vectores de V_n , y el conjunto de todas sus combinaciones lineales constituye un subespacio de V_n : el *espacio fila* de la matriz. Su dimensión es el *rango de filas*. De manera análoga se define el rango de columnas, y vale el resultado clásico: **el rango de filas coincide con el rango de columnas**, y a ese valor común se lo llama simplemente *rango* de la matriz.

Si las k filas de una matriz G son linealmente independientes, su espacio fila tiene dimensión k y contiene 2^k vectores. Ese espacio fila será, en la unidad siguiente, el código de bloque lineal (n, k) generado por G .

9.2. Operaciones elementales y forma sistemática

Las *operaciones elementales de fila* (intercambiar dos filas, y sumar una fila a otra; sobre $GF(2)$ no hay más, ya que el único escalar no nulo es el 1) no alteran el espacio fila de la matriz. Esto permite llevar la matriz a una forma canónica mediante eliminación gaussiana, con dos objetivos prácticos:

- **Detectar filas redundantes** y determinar así el rango verdadero de la matriz. Si al escalonar aparece una fila nula, es que una de las filas originales era combinación lineal de las otras, y la dimensión real del código es menor que la cantidad de filas de partida.
- **Obtener la forma sistemática**. Permitiendo además permutaciones de columnas, toda matriz de rango k puede llevarse a la forma

$$G = \begin{bmatrix} I_k & P \end{bmatrix},$$

donde I_k es la matriz identidad de $k \times k$ y P es una matriz de $k \times (n - k)$. Cuando la matriz generadora tiene esta forma, los primeros k símbolos de la palabra de código coinciden con el mensaje, y los $n - k$ restantes son los símbolos de paridad. Éste es el origen de la palabra “sistemático”, y la razón por la cual la codificación sistemática es tan cómoda: no hay que “extraer” el mensaje en recepción, ya está a la vista.

La permutación de columnas modifica el código (permuta las posiciones de los símbolos) pero no sus propiedades de distancia, de modo que el código resultante es *equivalente* al original y tiene la misma capacidad de corrección.

9.3. Espacio nulo de una matriz

Dada una matriz G de $k \times n$ y rango k , el conjunto de los vectores $\mathbf{u}$ de V_n que satisfacen

$$\mathbf{u} \cdot G^T = \mathbf{0}$$

es un subespacio de dimensión $n - k$, llamado *espacio nulo* de G . Cualquier matriz H de $(n - k) \times n$ cuyas filas formen una base de ese espacio nulo cumple entonces

$$G \cdot H^T = \mathbf{0}.$$

Con la forma sistemática $G = \begin{bmatrix} I_k & P \end{bmatrix}$, una elección posible es

$$H = \begin{bmatrix} P^T & I_{n-k} \end{bmatrix},$$

como puede verificarse directamente. Este par de matrices, G y H , es el objeto central de la unidad siguiente: la primera genera el código, la segunda lo verifica.

10. Cierre: qué se hace con todo esto

Cada herramienta construida aquí tiene un destinatario preciso en el resto del curso:

- El concepto de **espacio vectorial y subespacio** define lo que es un código de bloque lineal; las **matrices** G y H y la noción de **espacio nulo** dan el encoder y el cálculo del síndrome (Unidad 4).
- La **aritmética de polinomios sobre $GF(2)$** , y en particular la división con resto, permite representar las palabras de código como polinomios e implementar codificación y cálculo de síndrome con registros de desplazamiento (Unidad 6).
- La **factorización de $X^{2^m-1} + 1$ en polinomios mínimos** es lo que permite elegir el polinomio generador de un código BCH imponiendo cuáles potencias de α deben ser raíces de toda palabra de código (Unidad 7).
- La **aritmética en $GF(2^m)$** (suma con XOR, multiplicación con exponentes, búsqueda exhaustiva de raíces) es la que ejecutan literalmente el algoritmo de Berlekamp–Massey, la búsqueda de Chien y la fórmula de Forney (Unidades 7 y 8), y es lo que se implementa en RTL en el módulo de aritmética sobre $GF(2^8)$ del trabajo práctico final.

Dicho de otro modo: el resto del curso es, en buena medida, la aplicación sistemática de lo que se construyó en esta unidad.

Referencias

- [1] S. Lin y D. J. Costello, *Error Control Coding*, 2^a ed. Upper Saddle River, NJ: Pearson Prentice Hall, 2004. (Capítulo 2: *Introduction to Algebra*.)
- [2] E. Sanvicente, *Understanding Error Control Coding*. Springer Nature Switzerland AG, 2019. doi: 10.1007/978-3-030-05840-1. (Capítulo 2: *A First Look at Block Coders*; Capítulo 3: *RS and Binary BCH Codes*, Sección 3.3.)